

MAKALAH
SIMULASI IMPLEMENTASI CACHING STRATEGY,
REDIS/MEMCACHED CONCEPT, CDN, CACHE
INVALIDATION, DAN CACHE EVICTION PADA SISTEM WEB
BERBASIS LOCALHOST



Disusun oleh :
M. IQBAL TAWAKKAL (105841118723)

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH MAKASSAR 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga laporan praktikum Sistem Distribusi yang berjudul "**Simulasi Implementasi Caching Strategy, Redis/Memcached Concept, CDN, Cache Invalidation, dan Cache Eviction pada Sistem Web Berbasis Localhost**" dapat diselesaikan dengan baik.

Laporan ini disusun sebagai salah satu bentuk dokumentasi dan pertanggungjawaban atas kegiatan praktikum yang telah dilaksanakan. Praktikum ini bertujuan untuk mempelajari konsep caching yang merupakan salah satu teknologi penting dalam sistem distribusi modern. Melalui simulasi yang dibangun menggunakan aplikasi web berbasis localhost, penulis dapat memahami bagaimana data disimpan sementara pada cache untuk mempercepat proses akses data dan mengurangi beban pada database.

Dalam penyusunan laporan ini, penulis mempelajari berbagai konsep yang berkaitan dengan teknologi caching, seperti Cache Aside Strategy, Redis/Memcached sebagai cache layer, Content Delivery Network (CDN), Cache Invalidation, dan Cache Eviction. Selain itu, dilakukan pula pengujian terhadap mekanisme Cache Miss, Cache Hit, Cache Invalidation, dan Cache Eviction untuk mengetahui bagaimana cache bekerja dalam meningkatkan performa sistem.

Penulis menyadari bahwa laporan ini masih memiliki berbagai kekurangan baik dari segi penyajian maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang bersifat membangun demi penyempurnaan laporan ini di masa yang akan datang.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca mengenai penerapan teknologi caching dalam sistem distribusi.

Makassar, 2 Juni 2026

M. IQBAL TAWAKKAL

DAFTAR ISI

KATA PENGANTAR	2
BAB I PENDAHULUAN	4
1.1 Latar Belakang.....	4
1.2 Rumusan Masalah	5
1.3 Batasan Masalah	5
1.4 Tujuan Penulisan.....	5
1.5 Manfaat Penulisan.....	5
BAB II LANDASAN TEORI.....	6
2.1 Caching	6
2.2 Cache Aside Strategy	6
2.3 Redis dan Memcached.....	6
2.4 Content Delivery Network (CDN).....	7
BAB III PERANCANGAN SISTEM	8
3.1 Arsitektur Sistem.....	8
3.2 Alur Cache Miss	8
3.3 Alur Cache Hit.....	9
3.4 Alur Cache Invalidation	9
3.5 Alur Cache Eviction	10
3.6 Implementasi Program	10
3.7 Struktur Antarmuka (index.html)	10
3.8 Implementasi Tampilan (style.css).....	11
BAB IV HASIL DAN PEMBAHASAN.....	12
4.1 Pengujian Kondisi Awal	12
4.2 Pengujian Cache Miss	13
4.3 Pengujian Cache Hit.....	13
4.4 Pengujian Cache Invalidation.....	14
4.5 Pengujian Cache Eviction.....	15
4.6 Analisis Keseluruhan	
BAB V PENUTUP	17
5.1 Kesimpulan	17
5.2 Saran.....	17
DAFTAR PUSTAKA	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi telah mendorong penggunaan aplikasi berbasis web dalam berbagai bidang, seperti pendidikan, bisnis, pemerintahan, dan layanan publik. Seiring meningkatnya jumlah pengguna dan kebutuhan akan akses informasi yang cepat, sistem dituntut untuk mampu memberikan layanan yang responsif dan efisien. Namun, meningkatnya jumlah permintaan yang diterima oleh server dapat menyebabkan beban kerja yang tinggi, terutama apabila setiap permintaan harus mengambil data secara langsung dari database. Kondisi tersebut dapat mengakibatkan meningkatnya waktu respons sistem serta menurunkan kualitas layanan yang diberikan kepada pengguna.

Salah satu solusi yang banyak digunakan untuk mengatasi permasalahan tersebut adalah penerapan teknologi caching. Caching merupakan teknik penyimpanan data sementara yang bertujuan untuk mempercepat proses akses data yang sering digunakan. Dengan adanya cache, data tidak perlu selalu diambil dari database sehingga waktu yang dibutuhkan untuk menampilkan informasi kepada pengguna menjadi lebih singkat. Selain meningkatkan kecepatan akses data, caching juga dapat mengurangi beban kerja database dan penggunaan sumber daya server.

Dalam implementasinya, teknologi caching umumnya memanfaatkan sistem penyimpanan data berbasis memori seperti Redis dan Memcached. Kedua teknologi tersebut berfungsi sebagai cache layer yang berada di antara aplikasi dan database. Ketika suatu data sering diakses, data tersebut akan disimpan sementara pada cache sehingga permintaan berikutnya dapat dilayani tanpa harus mengakses database secara langsung. Konsep ini sangat penting dalam sistem distribusi karena mampu meningkatkan performa dan skalabilitas aplikasi.

Selain penggunaan cache untuk data dinamis, sistem modern juga memanfaatkan Content Delivery Network (CDN) untuk mempercepat distribusi file statis seperti HTML, CSS, JavaScript, gambar, dan video. CDN bekerja dengan menyimpan salinan file pada berbagai lokasi sehingga pengguna dapat mengakses file dari server yang paling dekat. Meskipun pada praktikum ini tidak menggunakan CDN secara langsung, konsep tersebut dapat disimulasikan melalui browser cache yang menyimpan file statis sehingga tidak perlu diunduh ulang setiap kali halaman dibuka.

Penerapan caching juga memerlukan mekanisme pengelolaan data yang baik. Salah satu mekanisme tersebut adalah Cache Invalidation, yaitu proses penghapusan atau pembaruan cache ketika data utama mengalami perubahan. Mekanisme ini penting untuk memastikan bahwa data yang tersimpan pada cache tetap sesuai dengan data yang ada pada database. Selain itu terdapat pula Cache Eviction, yaitu proses penghapusan data cache secara otomatis ketika masa berlaku cache telah habis atau ketika kapasitas penyimpanan cache mencapai batas tertentu. Kedua mekanisme tersebut berperan penting dalam menjaga konsistensi data serta efisiensi penggunaan memori.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan sebelumnya, maka rumusan masalah dalam praktikum ini dapat dirumuskan sebagai berikut:

1. Bagaimana konsep dasar caching diterapkan dalam sistem distribusi untuk meningkatkan performa aplikasi?
2. Bagaimana cara kerja Cache Aside Strategy dalam proses penyimpanan dan pengambilan data pada cache?
3. Bagaimana konsep Redis/Memcached dan CDN berperan dalam mempercepat akses data pada sistem?
4. Bagaimana mekanisme Cache Invalidation dan Cache Eviction digunakan untuk menjaga konsistensi dan efisiensi data cache?

1.3 Batasan Masalah

Pada praktikum ini pembahasan difokuskan pada simulasi implementasi caching berbasis localhost menggunakan aplikasi web sederhana. Simulasi hanya mencakup penerapan Cache Aside Strategy, konsep Redis/Memcached sebagai cache layer, simulasi CDN melalui browser cache, serta mekanisme Cache Invalidation dan Cache Eviction.

1.4 Tujuan Penulisan

untuk memahami konsep dan implementasi teknologi caching pada sistem distribusi, mempelajari cara kerja Cache Aside Strategy, memahami peran Redis/Memcached dan CDN dalam meningkatkan performa sistem, serta menganalisis mekanisme Cache Invalidation dan Cache Eviction dalam menjaga konsistensi dan efisiensi data yang tersimpan pada cache.

1.5 Manfaat Penulisan

Melalui praktikum dan penyusunan laporan ini diharapkan pembaca dapat memahami konsep dasar caching beserta manfaatnya dalam meningkatkan performa aplikasi. Selain itu, laporan ini dapat menjadi referensi pembelajaran mengenai implementasi Redis/Memcached, CDN, Cache Invalidation, dan Cache Eviction serta memberikan gambaran mengenai bagaimana teknologi caching digunakan untuk mengurangi beban database dan mempercepat proses akses data pada sistem distribusi.

BAB II

LANDASAN TEORI

2.1 Caching

Caching merupakan teknik penyimpanan data sementara yang bertujuan untuk mempercepat proses akses data yang sering digunakan. Dalam sebuah sistem, data yang tersimpan pada database umumnya membutuhkan waktu lebih lama untuk diakses dibandingkan data yang tersimpan pada cache. Oleh karena itu, ketika suatu data sering diminta oleh pengguna, sistem dapat menyimpannya pada cache sehingga permintaan berikutnya dapat dilayani dengan lebih cepat tanpa harus mengakses database kembali.

Penggunaan caching memberikan berbagai keuntungan, antara lain mengurangi beban kerja database, mempercepat waktu respons aplikasi, menghemat penggunaan sumber daya server, serta meningkatkan pengalaman pengguna. Saat ini hampir seluruh aplikasi berskala besar seperti Google, YouTube, Netflix, Facebook, dan Instagram menggunakan teknologi caching untuk meningkatkan performa layanan mereka.

2.2 Cache Aside Strategy

Cache Aside Strategy merupakan salah satu strategi caching yang paling banyak digunakan dalam pengembangan aplikasi modern. Pada strategi ini aplikasi akan memeriksa cache terlebih dahulu sebelum mengambil data dari database. Jika data ditemukan pada cache maka kondisi tersebut disebut Cache Hit. Sebaliknya, jika data tidak ditemukan maka terjadi Cache Miss dan aplikasi harus mengambil data dari database.

Setelah data berhasil diperoleh dari database, data tersebut kemudian disimpan kembali ke dalam cache sehingga permintaan berikutnya dapat dilayani lebih cepat. Strategi ini banyak digunakan karena mudah diterapkan, fleksibel, dan mampu mengurangi jumlah akses langsung ke database secara signifikan.

2.3 Redis dan Memcached

Redis dan Memcached merupakan teknologi penyimpanan data berbasis memori (in-memory storage) yang banyak digunakan sebagai cache layer pada aplikasi modern. Kedua teknologi ini bekerja dengan menyimpan data sementara di dalam RAM sehingga proses pembacaan data dapat dilakukan jauh lebih cepat dibandingkan membaca data dari database.

Redis memiliki fitur yang lebih lengkap karena mendukung berbagai struktur data seperti string, list, set, dan hash. Selain itu Redis juga mendukung mekanisme TTL (Time To Live) untuk mengatur masa berlaku cache. Sementara itu Memcached lebih sederhana dan ringan karena hanya berfokus pada penyimpanan data berbentuk key-value.

Dalam simulasi yang dibuat pada praktikum ini, Redis dan Memcached tidak digunakan secara langsung, melainkan direpresentasikan dalam bentuk cache layer yang berada di antara client dan database untuk memperlihatkan konsep dasar cara kerja teknologi tersebut.

2.4 Content Delivery Network (CDN)

Content Delivery Network atau CDN merupakan sekumpulan server yang tersebar di berbagai lokasi geografis dan digunakan untuk mendistribusikan file statis kepada pengguna dengan lebih cepat. CDN bekerja dengan menyimpan salinan file statis seperti HTML, CSS, JavaScript, gambar, dan video pada berbagai server sehingga pengguna dapat mengakses file dari lokasi yang paling dekat.

Penggunaan CDN dapat mengurangi latency, mempercepat waktu loading halaman web, mengurangi beban server utama, dan meningkatkan ketersediaan layanan. Pada simulasi ini konsep CDN direpresentasikan melalui browser cache yang menyimpan file statis sehingga file tidak perlu diunduh ulang setiap kali halaman dibuka.

2.5 Cache Invalidation

Cache Invalidation merupakan proses penghapusan atau pembaruan cache ketika data utama mengalami perubahan. Mekanisme ini sangat penting karena cache yang tidak diperbarui dapat menyebabkan pengguna menerima informasi yang sudah tidak sesuai dengan kondisi sebenarnya.

Sebagai contoh, apabila harga suatu produk berubah pada database, maka cache lama harus dihapus agar sistem dapat membuat cache baru yang berisi data terbaru. Dengan demikian konsistensi antara cache dan database dapat tetap terjaga.

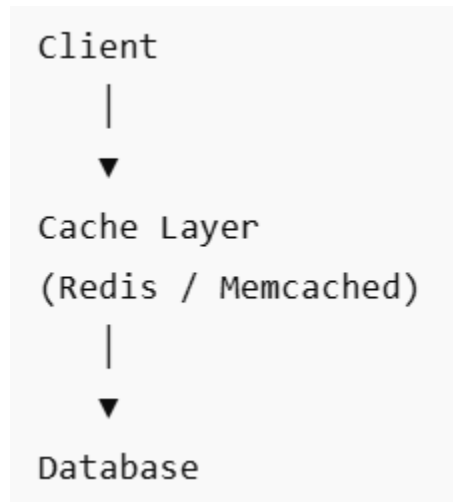
2.6 Cache Eviction

Cache Eviction merupakan proses penghapusan data cache yang sudah tidak diperlukan lagi. Penghapusan ini biasanya dilakukan secara otomatis berdasarkan aturan tertentu, seperti TTL (Time To Live) yang telah habis atau ketika kapasitas cache telah mencapai batas maksimum.

Tujuan utama Cache Eviction adalah menjaga efisiensi penggunaan memori dan memastikan bahwa cache hanya menyimpan data yang masih relevan. Dalam simulasi ini Cache Eviction dilakukan dengan menghapus cache setelah masa aktif tertentu sehingga sistem harus membuat cache baru pada permintaan berikutnya.

BAB III PERANCANGAN SISTEM

3.1 Arsitektur Sistem

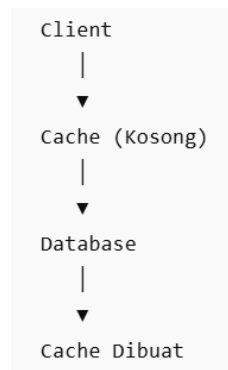


Penjelasan:

Arsitektur sistem simulasi caching terdiri atas tiga komponen utama yaitu Client, Cache Layer, dan Database. Client merupakan pengguna yang melakukan permintaan data ke sistem. Sebelum mengambil data dari database, sistem akan terlebih dahulu memeriksa apakah data tersedia pada cache. Jika data tersedia maka data akan langsung dikirimkan ke pengguna tanpa mengakses database. Sebaliknya, jika data tidak tersedia maka sistem akan mengambil data dari database kemudian menyimpannya ke cache untuk digunakan pada permintaan berikutnya.

Penerapan arsitektur ini bertujuan untuk mengurangi beban database dan meningkatkan kecepatan respons sistem. Pada simulasi ini, Cache Layer merepresentasikan konsep Redis atau Memcached yang umum digunakan pada aplikasi web modern.

3.2 Alur Cache Miss

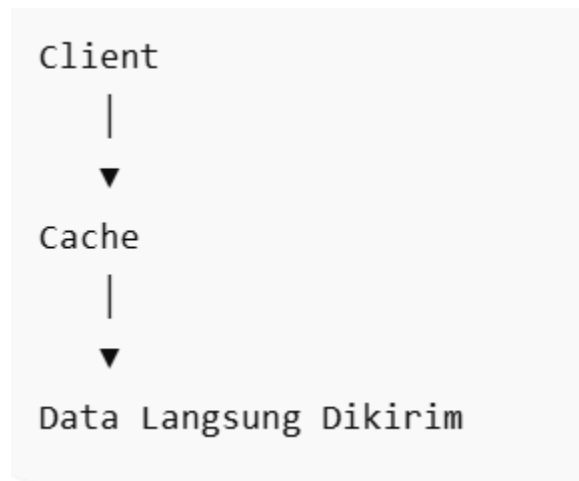


Penjelasan

Cache Miss terjadi ketika sistem tidak menemukan data yang diminta pada cache. Dalam kondisi ini sistem harus mengambil data langsung dari database. Setelah data berhasil diperoleh, sistem akan menyimpan data tersebut ke dalam cache agar dapat digunakan kembali pada permintaan berikutnya.

Proses ini membutuhkan waktu lebih lama dibandingkan Cache Hit karena database harus diakses terlebih dahulu.

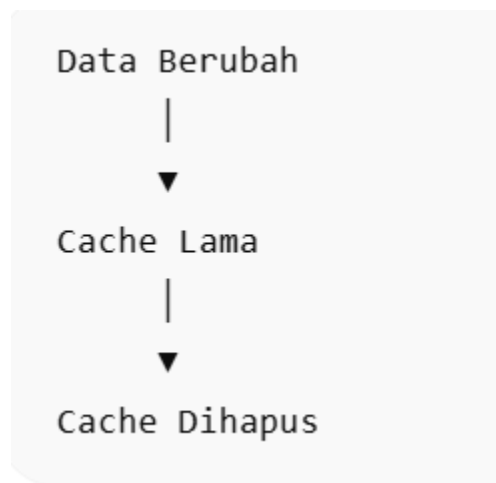
3.3 Alur Cache Hit



Penjelasan

Cache Hit terjadi ketika data yang diminta pengguna telah tersedia di dalam cache. Sistem tidak perlu lagi mengakses database sehingga data dapat langsung dikirimkan kepada pengguna. Kondisi ini memberikan waktu respons yang lebih cepat dan mengurangi beban kerja database.

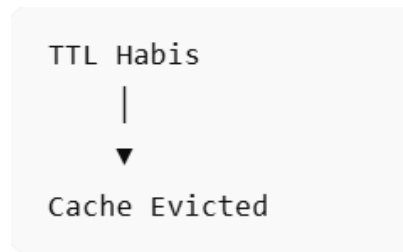
3.4 Alur Cache Invalidation



Penjelasan

Cache Invalidation merupakan proses penghapusan cache yang dilakukan ketika data pada database mengalami perubahan. Tujuan dari proses ini adalah memastikan bahwa pengguna tidak menerima data lama yang sudah tidak sesuai dengan kondisi sebenarnya. Setelah cache dihapus, sistem akan membuat cache baru saat permintaan data berikutnya dilakukan.

3.5 Alur Cache Eviction



Penjelasan

Cache Eviction adalah mekanisme penghapusan cache secara otomatis ketika masa aktif data atau Time To Live (TTL) telah habis. Mekanisme ini digunakan untuk menghemat penggunaan memori cache serta memastikan data yang tersimpan tetap relevan dan tidak terlalu lama.

3.6 Implementasi Program

```
<> JavaScript

const express = require("express");
const app = express();

app.use(express.static("public"));

app.listen(3000, () => {
  console.log("Server running at http://localhost:3000");
});
```

Penjelasan:

Kode di atas berfungsi untuk menjalankan server pada port 3000. Folder public digunakan untuk menyimpan file HTML, CSS, dan JavaScript yang akan ditampilkan kepada pengguna melalui browser.

3.7 Struktur Antarmuka (index.html)

```
HTML

<button onclick="cacheMiss()">
  Cache Miss
</button>

<button onclick="cacheHit()">
  Cache Hit
</button>

<button onclick="invalidateCache()">
  Invalidation
</button>

<button onclick="evictCache()">
  Eviction
</button>
```

Penjelasan:

Tombol-tombol tersebut digunakan untuk mensimulasikan empat kondisi utama dalam sistem caching, yaitu Cache Miss, Cache Hit, Cache Invalidation, dan Cache Eviction.

3.8 Implementasi Tampilan (style.css)

```
</> CSS

.card{
  background:white;
  padding:20px;
  border-radius:12px;
  box-shadow:0 2px 10px rgba(0,0,0,.08);
}

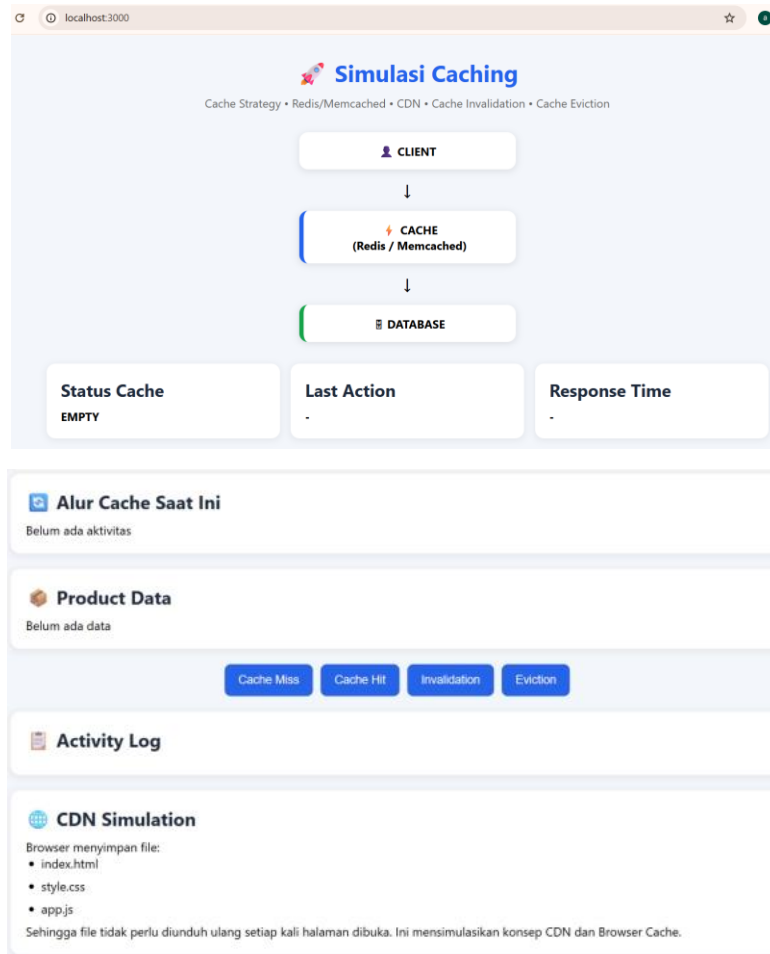
button{
  background:#2563eb;
  color:white;
  border:none;
  border-radius:8px;
}
```

Penjelasan:

Kode CSS digunakan untuk mempercantik tampilan aplikasi sehingga lebih mudah dipahami oleh pengguna saat melakukan simulasi pengujian caching.

BAB IV HASIL DAN PEMBAHASAN

4.1 Pengujian Kondisi Awal



Analisis

Pada kondisi awal sistem belum memiliki data yang tersimpan di dalam cache. Hal ini terlihat dari status cache yang masih menunjukkan kondisi EMPTY dan bagian Product Data yang belum menampilkan informasi produk. Activity Log juga masih kosong karena belum ada aktivitas yang dilakukan pengguna.

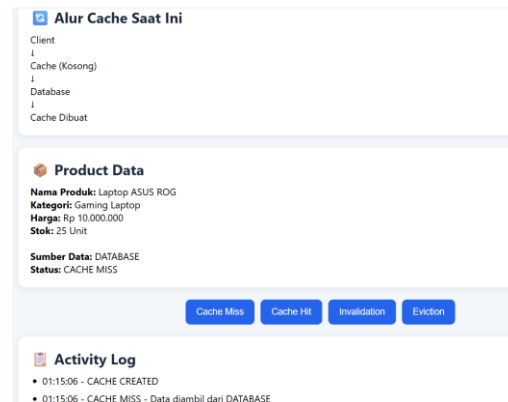
Kondisi ini menggambarkan keadaan awal sistem sebelum proses caching berlangsung. Pada tahap ini cache belum menyimpan data apa pun sehingga setiap permintaan data nantinya akan dianggap sebagai Cache Miss.

Hasil Pengujian

- Status Cache : EMPTY
- Data Produk : Belum tersedia
- Activity Log : Kosong

- Cache : Belum dibuat

4.2 Pengujian Cache Miss



Analisis

Pada pengujian ini tombol Cache Miss ditekan untuk mensimulasikan kondisi ketika cache belum memiliki data yang diminta. Sistem melakukan pemeriksaan cache terlebih dahulu dan menemukan bahwa data tidak tersedia.

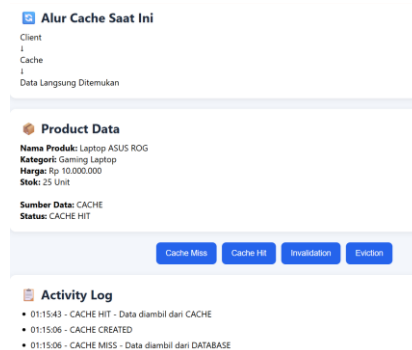
Karena cache kosong, sistem mengakses database untuk mengambil informasi produk berupa Laptop ASUS ROG. Setelah data berhasil diperoleh, sistem kemudian membuat cache baru dan menyimpan data tersebut ke dalam cache.

Activity Log menunjukkan aktivitas CACHE CREATED dan CACHE MISS sehingga membuktikan bahwa data berhasil diambil dari database dan cache berhasil dibuat.

Hasil Pengujian

- Sumber Data : Database
- Status : Cache Miss
- Cache Baru : Dibuat
- Database : Diakses

4.3 Pengujian Cache Hit



Analisis

Pada pengujian Cache Hit, sistem berhasil menemukan data yang dibutuhkan di dalam cache sehingga database tidak perlu diakses kembali. Data produk langsung dikirimkan kepada pengguna dari cache.

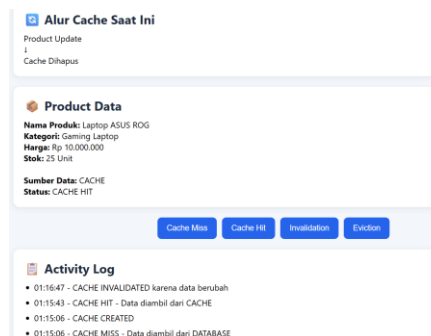
Proses ini menghasilkan waktu respons yang lebih cepat dibandingkan Cache Miss karena tidak terjadi proses pengambilan data dari database.

Activity Log menunjukkan aktivitas CACHE HIT yang menandakan bahwa sistem berhasil menggunakan cache sebagai sumber data utama.

Hasil Pengujian

- Sumber Data : Cache
- Status : Cache Hit
- Database : Tidak diakses
- Waktu Respons : Lebih cepat

4.4 Pengujian Cache Invalidation



Analisis

Pada pengujian ini dilakukan simulasi perubahan data. Ketika data berubah, cache lama harus dihapus agar sistem tidak menampilkan informasi yang sudah tidak sesuai.

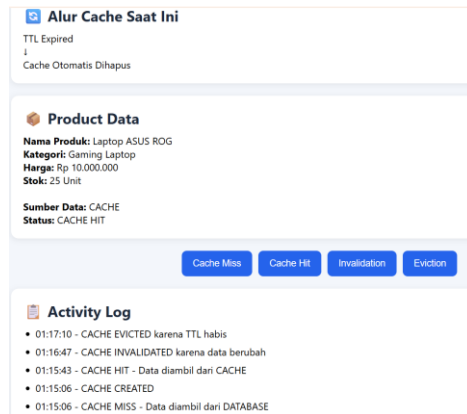
Proses penghapusan cache ini disebut Cache Invalidation. Setelah cache dihapus, permintaan berikutnya akan membuat cache baru berdasarkan data terbaru dari database.

Activity Log menampilkan pesan CACHE INVALIDATED karena data berubah, yang menunjukkan bahwa mekanisme invalidasi berjalan dengan baik.

Hasil Pengujian

- Data : Berubah
- Cache Lama : Dihapus
- Konsistensi Data : Terjaga
- Status : Cache Invalidated

4.5 Pengujian Cache Eviction



Analisis

Pada pengujian ini dilakukan simulasi berakhirnya masa aktif cache atau Time To Live (TTL). Setelah TTL habis, sistem secara otomatis menghapus cache yang tersimpan.

Mekanisme ini disebut Cache Eviction. Tujuannya adalah menghemat penggunaan memori dan menghapus data yang sudah tidak diperlukan lagi.

Activity Log menunjukkan pesan CACHE EVICTED karena TTL habis yang menandakan bahwa proses eviction berjalan dengan baik.

Hasil Pengujian

- TTL : Habis
- Cache : Dihapus Otomatis
- Status : Cache Evicted
- Database : Akan diakses kembali pada request berikutnya

4.6 Analisis Keseluruhan

Berdasarkan seluruh pengujian yang telah dilakukan, sistem berhasil mensimulasikan konsep Cache Aside Strategy, Redis/Memcached Concept, CDN Browser Cache, Cache Invalidation, dan Cache Eviction. Pengujian Cache Miss menunjukkan bahwa sistem harus mengakses database ketika data belum tersedia pada cache. Pengujian Cache Hit menunjukkan peningkatan performa karena data dapat langsung diambil dari cache tanpa mengakses database.

Selain itu, mekanisme Cache Invalidation berhasil menjaga konsistensi data ketika terjadi perubahan informasi pada database, sedangkan Cache Eviction berhasil menghapus data cache secara otomatis setelah masa aktifnya habis. Simulasi CDN juga menunjukkan bagaimana browser dapat menyimpan file statis seperti HTML, CSS, dan JavaScript sehingga tidak perlu diunduh ulang setiap kali halaman dibuka.

Hasil pengujian membuktikan bahwa penggunaan cache mampu meningkatkan efisiensi sistem, mempercepat waktu respons, dan mengurangi beban database sehingga performa aplikasi menjadi lebih optimal.

BAB V

PENUTUP

5.1 Kesimpulan

Berdasarkan hasil pengujian yang telah dilakukan, simulasi caching berhasil menunjukkan konsep Cache Aside Strategy, Redis/Memcached Concept, CDN Browser Cache, Cache Invalidation, dan Cache Eviction. Pengujian Cache Miss menunjukkan bahwa data diambil dari database ketika cache kosong, sedangkan Cache Hit menunjukkan bahwa data dapat diambil langsung dari cache sehingga waktu akses menjadi lebih cepat. Selain itu, Cache Invalidation dan Cache Eviction berhasil mendemonstrasikan proses penghapusan cache untuk menjaga konsistensi data dan efisiensi penggunaan memori. Dengan demikian, penggunaan cache dapat membantu meningkatkan performa aplikasi dan mengurangi beban database.

5.2 Saran

Simulasi ini dapat dikembangkan lebih lanjut dengan mengintegrasikan Redis atau Memcached yang sebenarnya serta menggunakan database nyata agar hasil pengujian lebih mendekati implementasi pada lingkungan produksi. Selain itu, fitur pengukuran waktu respons dapat ditambahkan untuk memperlihatkan perbedaan performa antara akses cache dan akses database secara lebih jelas.

DAFTAR PUSTAKA

1. Redis Labs. *Redis Documentation*. <https://redis.io/docs>
2. Memcached Project. *Memcached Documentation*. <https://memcached.org>
3. Mozilla Developer Network. *HTTP Caching*. <https://developer.mozilla.org>
4. Node.js Foundation. *Node.js Documentation*. <https://nodejs.org>
5. Tanenbaum, A. S. *Distributed Systems: Principles and Paradigms*. Pearson Education, 2017.
6. Sommerville, I. *Software Engineering*. Pearson, 2016.